

PROCESOS - HILOS

DEPARTAMENTO DE CIENCIAS E INGENIERÍA DE LA COMPUTACIÓN

UNIVERSIDAD NACIONAL DEL SUR

KMC © 2026

Operaciones de procesos

1. **Fork()** esta llamada al sistema permite crear un proceso hijo en la familia de sistemas operativos UNIX (Linux).

Formato general

pid_t fork(void);

#include … (agregar las librería necesarias para que pueda compilar. En el manual de ayuda indica las librerías que se deben incluir)

```
…
int main(){
    int pid;
    pid = fork();
    if pid >0 { // en el proceso padre
            …..
    }
    Else {
```

```
 if pid == 0 { // en el proceso hijo
                ……
        }
        else   // error al crear el proceso
}
…..
exit(0);
}
```

Operaciones de procesos

2. **Wait()** esta llamada al sistema permite esperar hasta que un proceso hijo termine su ejecución.

Formato general

pid_t wait(int *wstatus);

```
…
int main(){
    int pid;
    pid = fork();
    if pid >0 { // en el proceso padre
            …..
                wait (NULL);
    }
    Else {
```

```
 if pid == 0 { // en el proceso hijo
                    ……
          }
          else   // error al crear el proceso
}
…..
exit(0);
}
```



Actividad …

```
main (int argc, char ** argv)
{
        int nuevop = fork();
        int x = 5;
        if (nuevop == 0) {
                x += 5;
        }
        else {
                nuevop = fork();
                x += 10;
                if(nuevop>0) {
                        x += 5; }
                }
}
```

Responder

¿Cuántas variables x hay?

¿Cuál es el valor final de cada una?

¿Es posible identificar el proceso padre de los procesos hijos?

Actividad …

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
int main()
{
   int i;
   pid _t pid;
   pid = fork();
   if (pid == 0) {
          for (i = 0; i < SIZE; i++) {
                 nums[i] = 2*i;
                 printf("Num[i]: %d ",nums[i]);}
   } else if (pid > 0) {
                 for (i = 0; i < SIZE; i++)
                        printf("Padre: %d ",nums[i]);
          }
   exit(0);
}
```

Responder

¿Cuántos procesos se crean durante la ejecución?

¿Es posible identificar al proceso padre?

Operaciones de procesos

3. **execve()** esta llamada al sistema reemplaza la actual imagen del proceso por una nueva imagen ejecutable.

Formato General

int execve(const char *pathname, char *const argv[], char *const envp[]);

```
…
int main(){
    int pid;
    char *args[] = {”ps", "-ax", 0};
    char *env[] = { 0 };
    pid = fork();
    if pid >0 { // en el proceso padre
            …..
                wait (NULL);
    }
    else {
```

```
 if pid == 0 { // en el proceso hijo
                 ……
                execve("/bin/ps", args, env);
                printf(“Error …. \n”);
          }
          else   // error al crear el proceso
}
…..
exit(0);
}
```

Operaciones de procesos

4. **exec()** esta familia de funciones reemplaza la actual imagen del proceso por una nueva imagen ejecutable.

   Las funciones de librería más utilizadas son execlp y execv.

Formato General

int execlp(const char *file, const char *arg, ... /* (char *) NULL */);

```
…
int main(){
    int pid;
    pid = fork();
    if pid >0 { // en el proceso padre
            …..
                wait (NULL);
    }
    else {
```

```
  if pid == 0 { // en el proceso hijo
                 ……
                execlp("ps", “ps”, “ax”, NULL);
                printf(“Error …. \n”);
          }
          else   // error al crear el proceso
}
…..
exit(0);
}
```



ACTIVIDAD DE CLASE

7 DE SEPTIEMBRE

KMC © 2026

Operaciones de hilos

1. **Pthread_create** esta es una función que permite crear un hilo.

Formato General

int pthread_create (pthread_t *thread, pthread_attr_t *attr, void *(*start_routine) (void *), void *arg)

Parámetros

thread – es un identificador único para el nuevo hilo (thread) devuelto por la rutina.

attr – se utiliza para especificarle atributos al nuevo hilo. Como recomendación utilicen NULL, para mantener la configuración que viene inicializada.

start_routine - es la rutina que va a ejecutar el nuevo hilo.

arg – es un parámetro que se le puede pasar a la rutina a ejecutar.

Operaciones de hilos

Pthread_create – Formas de definición

Identificar hilo

- pthread_t *hilos; // para definir varios hilos como un arreglo, puede traer problemas.
- pthread_t hilo; // para definir un hilo
- pthread_t hilo[10]; // para definir un arreglo de 10 hilos

Rutina de ejecución

```
void *mythread(void *data);
(main)
pthread_create(&hilos[i], NULL, mythread, NULL);
        …
void *mythread (void *data){
                …. // código
        }
```

Operaciones de hilos

Pthread_create – Formas de definición

```
Void lecto(void); // definición de la función de hilo

Pthread_t  reader;
    ...
pthread_create(&reader, NULL, (void *)&lecto, NULL);
```

Operaciones de hilos

2. **Pthread_exit** esta es una función indica que el hilo ha terminado su ejecución.

Formato General

void pthread_exit (void *retval)

Por ejemplo: **Pthread_exit(NULL);**

Operaciones de hilos

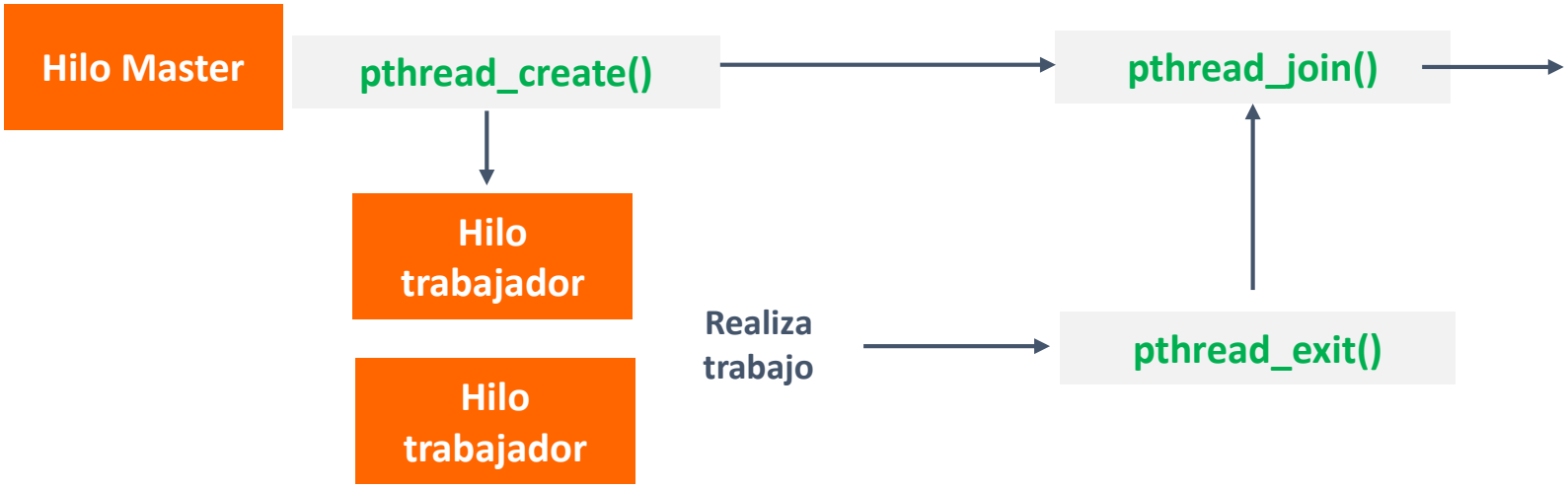
3. **Pthread_join** esta es una función que se utiliza para esperar que termine la ejecución de un hilo que ha sido creado por el proceso.

Formato General

int pthread_join (pthread_t thread, void **retval)

Por ejemplo: **Pthread_join (hilos[i], NULL);**

Operaciones de hilos



Actividad …

- ¿Cuál es el comportamiento de este proceso?
- ¿Siempre tiene el mismo resultado?, ¿es determinístico?
- ¿Está bien diseñado?

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
void *mihilo(void *data);

int main()
{
   int i;
   pid _t pid;
   pthread_t  hilo[SIZE];

   for (i = 0; i < SIZE; i++)
      pthread_create(&hilo[i], NULL, mihilo, (void *) (intptr_t)i);
   for (i = 0; i < SIZE; i++)
   printf("Num[i]: %d ",nums[i]);

   exit(0);
   }
```

```
void *mihilo(void *data){
   int j =  (intptr_t) data;
   nums[j] = 2*j;
   Printf("Num[i]: %d ",nums[j]);
}
```

Para compilar se debe incluir la librería lpthread en la línea de comando, sino se la incluye no les va a compilar.
Por ejemplo:

```
gcc –lpthread –o ej ejemplo.c
```

Para Recordar

"Dado que la CPU conmuta rápidamente entre un proceso y otro, la velocidad a la que un proceso ejecuta sus cálculos no es uniforme y tal vez ni siquiera sea reproducible si se ejecutan los mismos procesos de nuevo. …" (para complementar pueden leer el ejemplo que presenta)

Extraído del libro Sistemas Operativos Modernos, Andrew Tanenbaum (Cap. 2 de la 3er. o 4ta. Ed)

Actividad sobre Procesos

Son iguales?

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
int main()
{
   int i;
   pid _t pid;
   pid = fork();
   if (pid == 0) {
   for (i = 0; i < SIZE; i++) {
   nums[i] = 2*i;
   printf("Num[i]: %d ",nums[i]);}
   } else if (pid > 0) {
   for (i = 0; i < SIZE; i++)
   printf("Padre: %d ",nums[i]);
   }
exit(0);
}
```

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
int main()
{
   int i;
   pid_t pid;
   pid = fork();
   if (pid == 0) {
   for (i = 0; i < SIZE; i++) {
   nums[i] = 2*i;
   printf("Num[i]: %d ",nums[i]);}
   } else if (pid > 0) {
   wait (NULL);
   for (i = 0; i < SIZE; i++)
   printf("Padre: %d ",nums[i]);
   }
exit(0);
}
```

Actividad sobre Procesos

Son iguales?

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
int main()
{
   int i;
   pid _t pid;
   pid = fork();
   if (pid == 0) {
   for (i = 0; i < SIZE; i++) {
   nums[i] = 2*i;
   printf("Num[i]: %d ",nums[i]);}
   } else if (pid > 0) {
   for (i = 0; i < SIZE; i++)
   printf("Padre: %d ",nums[i]);
   }
return 0;
}
```

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
int main()
{
   int i;
   pid_t pid;
   pid = fork();
   if (pid == 0) {
   for (i = 0; i < SIZE; i++) {
   nums[i] = 2*i;
   printf("Num[i]: %d ",nums[i]);}
   } else if (pid > 0) {
   for (i = 0; i < SIZE; i++)
   printf("Padre: %d ",nums[i]);
   wait (NULL);
   }
return 0;
}
```

Actividad sobre Hilos

```
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
void *mihilo(void *data);

int main()
{
   int i;
   pid _t pid;
   pthread_t  hilo[SIZE];

   for (i = 0; i < SIZE; i++)
      pthread_create(&hilo[i], NULL, mihilo, (void *) (intptr_t)i);
   for (i = 0; i < SIZE; i++) {
      pthread_join(hilo[i], NULL);
      printf("Num[i]: %d ",nums[i]);
   }
   exit(0);
   }
```

```
void *mihilo(void *data){
   int  j = (intptr_t) data;
   nums[j] = 2*(j);
   Printf("Num[i]: %d ",nums[j]);
}
```

- **¿Es el mismo comportamiento que el primer caso ?**

Actividad – Procesos e Hilos

Para pensar ….

Cuál es la política al crear un proceso hijo en un proceso multihilado

